Preguntas de examen PSyD

Daniel Báscones García

3 de noviembre de 2026

Índice

1. Introducción y aviso **1**

2. Preguntas **1**

1. Introducción y aviso

En las siguientes páginas, se muestran **ejemplos** de preguntas de examen. Esto no quiere decir, ni que todas las preguntas vayan a salir de aquí (seguramente no), ni que sean refritos o adaptaciones de las mismas. Los contenidos que entran en examen incluyen tanto la teoría de los documentos teóricos, como la de las prácticas, y puede entrar cualquier cosa de aquellos siempre y cuando se haya dado en clase.

Así pues, esta colección sirve solo como referencia para entender la profundidad y dificultad que se puede esperar del examen, pero en ningún caso sustituye su resolución a estudiar la materia.

2. Preguntas

1. Complete la siguiente función:

```
/**
 * @param value es un puntero al dato que se cambia
 * @param byte_index es un índice (0-3) que indica el byte a cambiar
 * @param byte_value es el valor que hay que cambiar
 * @example Por ejemplo, si *value=0xabababab, byte_index=2,
 *          y byte_value=0xcd, el resultado será, que tras la llamada,
 *          *value=0xabcdabab
 */
static inline void set_byte(uint32_t * value, uint32_t byte_index, uint8_t
    byte_value) {

}
```

Solución:

```
static inline void set_byte(uint32_t * value, uint32_t byte_index, uint8_t
    byte_value) {
    *value &= ~(0xff << (byte_index * 8));
    *value &= (((uint32_t) byte_value & 0xff) << (byte_index * 8));
}
```

Table 39. CRC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	CRC_DR	DR[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x04	CRC_IDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDR[7:0]							
	Reset value																									0	0	0	0	0	0	0	0
0x08	CRC_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REV_OUT	REV_IN[1:0]		POLYSIZE[1:0]		Res.	Res.	RESET
	Reset value																									0	0	0	0	0			0
0x10	CRC_INIT	CRC_INIT[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x14	CRC_POL	POL[31:0]																															
	Reset value	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1	1	1	0	1	1	0	1	1	0	1	1	1

Figura 1: Tabla de registros del periférico CRC

2. El periférico `CRC` (ver Figura 1) sirve para generar códigos de comprobación que verifican la integridad de los datos. Su acceso se realiza a partir de la dirección `0x40023000`. Escriba el código C necesario como para inicializar el registro `CR` sin cambio de orden de datos (bits `REVx` a cero), ancho de 8 bits (`10b` en `POLYSIZE`) y forzando un reseteo (`RESET` a 1, este bit se limpia solo tras activarlo). Tenga en cuenta que no se permite un acceso directo a la dirección de memoria, sino que hay que crear una estructura de datos para el periférico.

   Solución:

```c
#define CRC_BASE_ADDR 0x40023000

typedef struct
{
    volatile uint32_t DR;
    volatile uint32_t IDR;
    volatile uint32_t CR;
    volatile uint32_t INIT;
    volatile uint32_t POL;
} CRC_TypeDef;

#define CRC ((CRC_TypeDef *) CRC_BASE_ADDR)

static inline void CRC_SET() {
    //         REV_OUT     REV_IN      POLYSIZE       RESET
    CRC->CR = (0 << 7) | (0 << 5) | (0b10 << 3) | (1);
}
```

3. ¿Qué efecto produce en la compilación calificar un puntero a un registro con la palabra clave `volatile` en lenguaje C?

   Solución: *Al nombrarlo como `volatile`, le decimos al compilador que su valor podría cambiar por fuentes ajenas al programa (por ejemplo el propio periférico). Por tanto, no debería realizar optimizaciones basadas en suposiciones sobre su valor, como eliminar bucles o reutilizarlo desde registros, leyéndolo/escribiéndolo siempre que lo necesite directamente en*

memoria.

4. Escriba una función en C `void gpio_set_mode(GPIO_TypeDef *GPIOx, uint32_t pin, uint32_t mode)` que configure de forma atómica a nivel de software el modo de un pin en el registro `MODER` (donde cada pin ocupa 2 bits). Asegúrese de limpiar primero la configuración previa mediante máscaras binarias antes de aplicar el nuevo modo

   Solución:

```
static inline void gpio_set_mode(GPIO_TypeDef *GPIOx, uint32_t pin,
    GPIO_Mode_t mode) {
    uint32_t temp = GPIOx->MODER;
    temp &= ~(0b11U << (pin * 2));
    temp |= ((uint32_t)mode << (pin * 2));
    GPIOx->MODER = temp;
}
```

5. En el microcontrolador STM32F723, el controlador de interrupciones NVIC maneja registros vectorizados como `ISER` (Interrupt Set Enable Register), definidos dentro del `struct` del periférico como un array `volatile uint32_t ISER[8]` para cubrir hasta 256 líneas de interrupción. Explique matemáticamente cómo se traduce un número global de línea de interrupción `IRQn` (por ejemplo, 71) a un índice dentro del array `ISER[]` y a una posición de bit dentro de ese elemento de 32 bits.

   Solución: *Al ser cada elemento del array una palabra de 32 bits, las línea de interrupción se distribuyen en bloques contiguos de 32 líneas.*

   El índice del registro es la parte entera de la división por 32: $\lfloor N/32 \rfloor$. *En operaciones de desplazamiento de bits equivale a* $N \gg 5$. *En nuestro ejemplo,* $71 \gg 5 = 2$ *(corresponde a* `ISER[2]` *).*

   La posición del bit dentro del registro de 32 bits es el resto de la división: $N \pmod{32}$. *En operaciones bit a bit equivale a* $N \,\&\, 0x1F$ *(*$N \,\&\, 31$*).*

6. Dado el siguiente *linker script*:

```
MEMORY
{
    RAM (xrw)    : ORIGIN = 0x20000000, LENGTH = 256K
    FLASH (rx)   : ORIGIN = 0x08000000, LENGTH = 512K
}

SECTIONS
{
    .isr_vector : {
        KEEP(*(.isr_vector))
    } >FLASH

    .text : {
        *(.text*)
        *(.rodata*)
    } >FLASH

    .rodata :
    {
        *(.rodata)
        *(.rodata*)
    } >FLASH

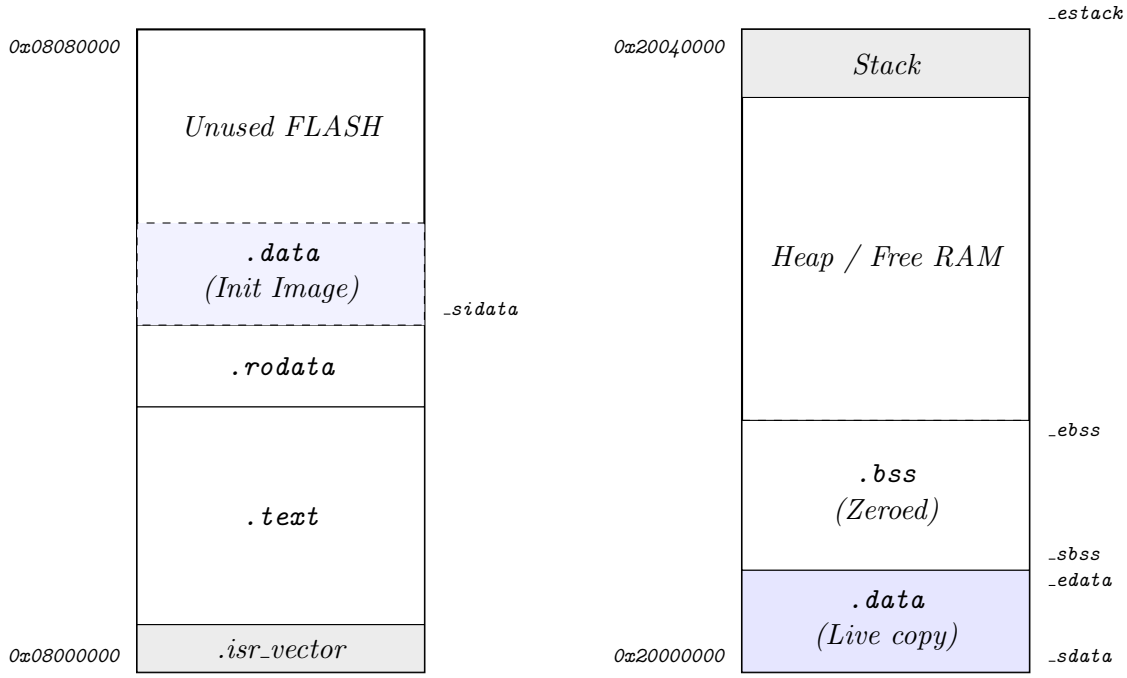
    _sidata = LOADADDR(.data);

    .data :
```

```
27      {
28          _sdata = .;         /* Inicio de datos en RAM */
29          *(.data)            /* .data sections */
30          *(.data*)           /* .data* sections */
31          _edata = .;         /* Fin de datos en RAM*/
32
33      } >RAM AT> FLASH
34
35      .bss : {
36          _sbss = .;
37          *(.bss*)
38          _ebss = .;
39      } > RAM
40
41      _estack = ORIGIN(RAM) + LENGTH(RAM);
42  }
```

- Realice un esquema del mapa de memoria de las diferentes secciones.
- Indique quién es el responsable de inicializar la memoria RAM, dónde reside su código, y cómo se llega a ejecutar.

Solución:



El encargado de inicializar la RAM es la función `Reset_Handler` *que se ejecuta lo primero al arrancar el procesador. Esta función se ubica en* `.text`*, y su dirección de memoria se encuentra en un vector predefinido de* `.isr_vector`*. Tras el arranque, se consulta esta dirección, y se salta a la misma, disparando el comportamiento mencionado (copiar los datos* `.data` *y limpiar* `.bss`*.*

7. Compare el funcionamiento del modelo de E/S por encuesta (*polling*) frente a las interrupciones. Analiza las ventajas y desventajas de cada uno en relación con la eficiencia de la CPU, latencia de respuesta y complejidad del código.

   Solución: *La E/S por polling implica estar continuamente, en un bucle, revisando los registros y periféricos para observar si se han producido interrupciones. La ventaja es que se puede mirar en momentos concretos, la desventaja es que es ineficiente, pues se pierde tiempo consultando en numerosas ocasiones donde no hay nada preparado. La latencia de respuesta, además, depende de la frecuencia de consulta. El código es muy sencillo si sólo*

se consulta en una ubicación, pero muy complejo si se necesita consultar en diferentes ubicaciones.

Por contra, la E/S basada en interrupciones nos permite desacoplar esta gestión, haciendo que el hilo principal se dedique a otras tareas, y únicamente se atiendan las interrupciones, de manera inmediata, cuando son avisadas. Esto aligera la carga computacional en la CPU y permite un código modular, con el sobrecoste de una función extra por cada interrupción.

8. ¿Qué datos contiene la tabla de vectores de un procesador? ¿En qué contexto y cómo se utilizan?

   Solución: *Contiene punteros a todas las ISRs que tratan las diversas interrupciones del procesador. Al dispararse una interrupción en una línea concreta, el puntero correspondiente se extrae, obteniéndose la dirección de comienzo de la ISR. El procesador guarda el estado del procesador, y salta a esa parte del código para atender la interrupción. Tras terminar, vuelve al punto desde donde se interrumpió recuperando el estado guardado.*

9. ¿Qué tareas básicas ha de realizar la rutina de inicialización `Reset_Handler` antes de saltar a `main`?

   Solución: *Inicializar el stack pointer, configurar algunos elementos básicos del sistema (reloj/memoria), y finalmente inicializar los datos de memoria (secciones `.data` y `.bss`) antes de saltar.*

10. A la hora de configurar las rutinas de tratamiento de interrupción (ISRs), se utilizan los siguientes fragmentos de código:

```
//1)
    . section . isr_vector , " a ", % progbits
    . type g_pfnVectors , %object

    g_pfnVectors :
        . word _estack /* 0 x00 : Pila inicial */
        //...
        . word USART1_IRQHandler
        //...

//2)
    . weak USART1_IRQHandler
    . thumb_set USART1_IRQHandler , Default_Handler

//3)
    void USART1_IRQHandler ( void ) {
        //...
    }
```

   Indica qué función cumple cada uno de ellos.

   Solución:

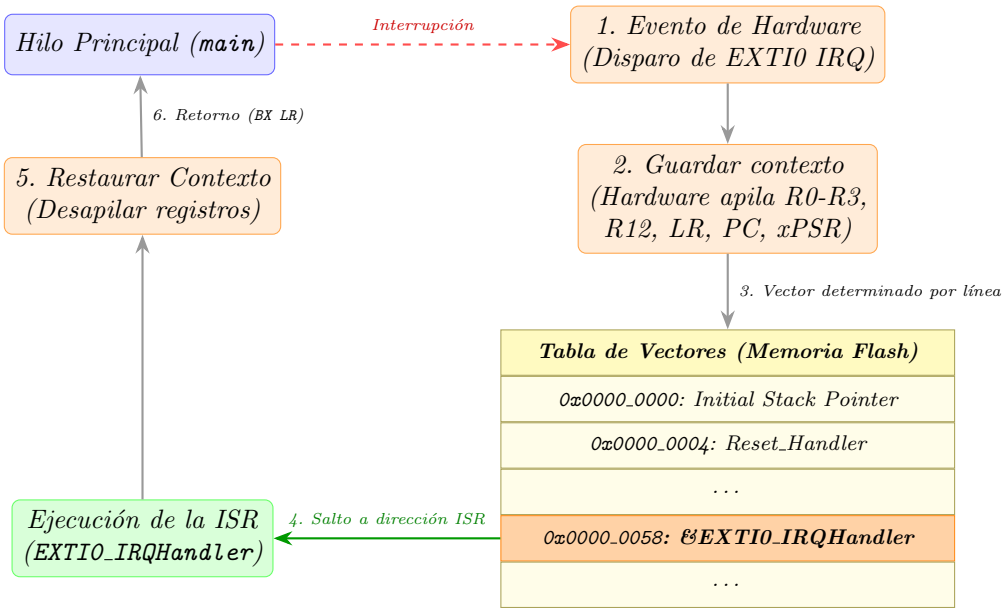
   a) *Inicializa la tabla de vectores, colocando cada puntero a ISR en las posiciones indicadas por sus líneas de interrupción correspondientes.*

   b) *Asigna, por defecto, la rutina `Default_Handler` para hacerse cargo de la interrupción en cuestión.*

   c) *Reescribe la ISR para que, al estar definida, sustituya el enlazado débil (`.weak`) anterior y se utilice automáticamente, en lugar del `Default_Handler`.*

11. Indica, preferiblemente con un diagrama, el flujo de ejecución cuando una ISR interrumpe el hilo principal de ejecución. En particular, señala qué ocurre con los registros y la tabla de vectores del procesador.

Solución: *La siguiente figura muestra el diagrama correspondiente:*



12. Explica las diferencias entre los `Callbacks` frente a gestionar ISRs directamente.

    Solución: *Utilizando directamente ISRs, tenemos entremezclado el código de gestión con la lógica de nuestro programa. Esto hace que sea menos portable (por ejemplo, al cambiar de arquitectura con diferentes ISRs) y menos flexible (sólo podemos atender las ISR desde un único lugar).*

    Los callbacks ofrecen la posibilidad de registrar múltiples funciones que serán llamadas tras una ISR para informar de la actualización del periférico o evento correspondiente, desacoplando la capa de gestión hardware de la capa software, y permitiendo mayor protabilidad.

13. Explique por qué en microcontroladores modernos la señal de reloj de cada periférico se encuentra desactivada por defecto tras un reinicio. ¿Cómo se gestiona su activación, y qué ocurre si se intenta acceder a los registros de un periférico cuyo reloj no ha sido habilitado previamente?

    Solución: *Al tener tantos periféricos en un microcontrolador moderno, mantener todos activos todo el tiempo sería ineficiente desde el punto de vista energético. Se aplican técnicas que aíslan los relojes de cada periférico, haciendo que su consumo sea cero en caso de no estar activos. Por tanto, en el caso de intentar acceder a un registro (especialmente en escritura), estos accesos no tendrán efecto, o devolverán datos basura.*

14. ¿Qué es un *Watchdog* y qué función tiene dentro de un procesador?

    Solución: *Un* Watchdog *es un temporizador que se reinicia cada cierto tiempo por un evento de procesador. Si pasa un tiempo sin reiniciarse, se acabará disparando y ejecutando una acción. Por ejemplo, es común utilizarlo para verificar que el procesador no se queda atascado en un bucle infinito, reiniciándose al cambiar el PC de valor, y disparándose en caso de que no cambie.*

15. ¿Qué sucede al intentar modificar los registros de control de un periférico (por ejemplo `GPIOA`) si no se ha activado previamente su reloj en el registro `RCC` correspondiente? (En este caso, `RCC_AHB1ENR`)

    Solución: *Como el periférico y sus registros no están recibiendo señal de reloj, no actualizarán sus valores y por tanto las escrituras habrán sido en vano.*

16. Si la frecuencia de reloj de entrada del periférico `USART6` es de 16 MHz y se requiere una velocidad de 115200 baudios ¿qué valor (aproximadamente) debe cargarse en el registro `USART6_BRR` ?

    Solución: *Para reducir la velocidad de reloj de entrada a la objetivo (los baudios) debemos utilizar un divisor de reloj. En este caso, el valor que le damos está en el registro `BRR`. El valor v_{BRR} necesario será:*

$$v_{BRR} = \frac{f_{in}}{baudrate} = \frac{16 \cdot 10^6}{115200} \approx 139$$

17. Explica cómo se consiguen transmitir datos por una `UART`. ¿Cómo se sincronizan emisor y receptor?

    Solución: *La `UART` tiene, entre otras configuraciones, la velocidad a la que trabaja. Es imprescindible que emisor y receptor tengan la misma velocidad de operación para sincronizarse. En cuanto a los datos, se transmiten paquetes de 7 u 8 bits (más paridad si es requerido), delimitados con marcadores de start y stop para identificarlos.*

18. En un `GPIO`, los pines se pueden configurar en varios modos: entrada, salida, modo alternativo y modo analógico. ¿Qué funcionalidad nos ofrece el modo alternativo, y cómo se utiliza?

    Solución: *El modo alternativo permite conectar el `GPIO` con uno de múltiples periféricos que lo comparten. Utilizando este modo les podemos dar salida al exterior y conectarlos con otros elementos externos al procesador, como por ejemplo conectar la `UART` con el ordenador. .*

19. El gestor de interrupciones externas `EXTI` tiene sólo unas pocas líneas (en concreto 16), muchas menos que la totalidad de pines de los que dispone el procesador (16 pines por puerto, en 10 puertos diferentes). ¿Cuántos pines puedo utilizar simultáneamente para interrumpir al procesador? ¿Qué restricciones existen?

    Solución: *`EXTI` multiplexa los pines de entrada, colocando todos los pines "0"(PA0, PB0, PC0,...) en la línea `EXTI0`, y así sucesivamente. Por tanto, el máximo de pines utilizables para interrumpir es de 16. La restricción existente es que nunca podremos utilizar dos pines con el mismo número (por ejemplo PA0 y PB0) a la vez, ya que comparten línea. Sí es posible sin embargo utilizar todos los pines de un puerto a la vez (PA0, PA1, PA2...) ya que cada uno va a una línea diferente.*

20. Asumiendo una frecuencia de entrada de 16MHz, calcule los valores de los registros de *preescalado* y *auto-reload* para conseguir un timer que se dispare cada 10 milisegundos.

    Solución: *Tenemos que conseguir una división de 160000 sobre la frecuencia original. El preescalado reduce la frecuencia de entrada, y el auto-reload es quien controla cada cuántos ciclos se dispara la interrupción. Por ejemplo, podemos configurar el preescalado a 16, y el auto-reload a 10000. Los valores exactos para lograr esta división serían de 15 y 9999, ya que el '0' cuenta como un ciclo aparte.*

21. Defina las variables y funciones necesarias para implementar un *ticker* de sistema que mida el tiempo en milisegundos mediante llamadas a una función `GetTick`. Asuma que el periférico ya está configurado para generar las interrupciones, y que la ISR que lo gestiona se llama `Systick_Handler`.

```
1 volatile uint32_t msTicks;
2
3 static uint32_t inline GetTick (void) {
4     return msTicks;
```

```
}

void SysTick_Handler (void) {
    msTicks++;
}
```

22. Compara las fuentes de reloj internas (HSI, LSI) con las externas por cristal de cuarzo (HSE, LSE) en términos de precisión, y consumo energético. ¿Por qué utilizaríamos un oscilador de cuarzo de 32,768 kHz para el reloj en tiempo real (RTC)?

    Solución: *Las fuentes internas son poco precisas, y se generan mediante osciladores basados en inversores o condensadores. Lo bueno es que están integradas y no hace falta hardware externo. Las fuentes externas basadas en cristal de cuarzo son mucho más precisas, pero ocupan espacio en la PCB. En cuanto al consumo, la principal diferencia radica en la frecuencia, a más frecuencia consumirá más, independientemente de si es externa o interna. El oscilador del RTC sería de 32KHz para ser de baja frecuencia y consumir menos, y de cuarzo para ser preciso y mantener correctamente la cuenta del tiempo.*

23. Describe la arquitectura física del bus I2C (líneas y funcionamiento). Explica la secuencia de pasos (a nivel lógico) que debe seguir un maestro para realizar una operación de lectura de un registro en un dispositivo esclavo.

    Solución: *Un bus I2C tiene dos líneas: reloj y datos. El maestro controla la señal de reloj y, dependiendo de la fase de la comunicación, los datos serán controlados también por maestro o esclavo. A nivel lógico, para leer un registro se manda primero la dirección del esclavo (en modo escritura), seguido del registro a leer. A continuación, se manda de nuevo la dirección del esclavo (en modo lectura), el cual responderá con el dato.*

24. En el control de periféricos mediante el bus I2C, el periférico no puede iniciar una transacción. Explique en este contexto si se pueden, y cómo, tratar las interrupciones.

    Solución: *Sí se pueden tratar, pero añadiendo líneas de interrupción externa. A través de éstas se disparará una interrupción que, a nivel de código, está atada a un periférico concreto. A partir de ese momento se hará consulta mediante una lectura I2C para ver qué información ha preparado el periférico.*

25. Explique las principales diferencias entre I2C y SPI, en cuanto a líneas utilizadas, direccionamiento de periféricos, e interrupciones.

    Solución: *I2C utiliza únicamente una línea de reloj y datos, mientras que SPI utiliza una línea de reloj, y dos de datos (MOSI y MISO) que permiten transferencias full-duplex simultáneas. SPI utiliza adicionalmente una línea CS por periférico, que permite seleccionar en cada momento el periférico activo, mientras que I2C direcciona a nivel lógico enviando la dirección por el bus. Ninguno de los dos controla interrupciones, que deben alimentarse desde líneas dedicadas y ajenas al protocolo.*

26. Explica por qué es obligatorio borrar antes de escribir un dato en memoria `FLASH`.

    Solución: *La memoria Flash solo permite pasar de un estado a otro (de 1 a 0). El borrado configura todo a 1, y al escribir se van poniendo ceros en las posiciones indicadas. Por construcción y debido a limitaciones tecnológicas, no se pueden escribir unos, así que la única opción es borrarla antes.*

27. Al almacenar datos de configuración en memoria no volátil, ¿cuál es el propósito de incluir un patrón o *magic number*? Describe el flujo lógico durante el arranque para decidir si los datos leídos son válidos o deben cargarse los valores por defecto.

Solución: *El patrón se incluye para poder saber si los datos que se leen han sido configurados a propósito, o son residuales de algún otro programa, o incluso datos vacíos por ser la primera vez que se inicia el dispositivo. Al arrancar, se comprueban unas posiciones de memoria predefinidas donde, si se encuentra el patrón, se entenderá que los datos que vienen a continuación son válidos. De lo contrario se cargan los valores por defecto.*

28. Explica la diferencia entre un planificador multitarea cooperativo y uno expropiativo. Analiza qué ocurre en un sistema cooperativo si una de las tareas entra en un bucle prolongado o bloqueante.

    Solución: *En un planificador cooperativo, cada tarea debe, manualmente, ceder el control (terminar). En uno expropiativo, un gestor (o kernel en un RTOS) se encarga de cambiar las tareas en ejecución incluso si éstas no han terminado voluntariamente. Esto permite que, si una se queda çolgada", el sistema la pueda sustituir por otra y no se pare la ejecución del resto. Esto no ocurre en multitarea cooperativa, donde si una tarea se bloquea, bloqueará también al resto hasta que se desbloquee, o indefinidamente si no lo hace,*

29. En un RTOS, explica la relación de las interrupciones `PENDSV` y `SysTick` con el cambio de tarea.

    Solución: *Cada cierto tiempo, la interrupción de* `Systick` *se dispara. En ese momento, se analizan las tareas pendientes de ejecutar y se determina cuál será la siguiente en ejecutar. En caso de existir (si ninguna tarea tiene que ejecutarse, el sistema seguiría a la espera), se activa la interrupción* `PENDSV` *de cara a que lance la tarea correspondiente al activarse. Esta interrupción tiene la prioridad mínima, por lo que se atenderán primero todas las demás ISR pendientes (para no quedarse a la espera) y finalmente la tarea correspondiente entrará en ejecución.*

30. Explique, en el contexto de un RTOS, las diferentes pilas o *Stacks* que existen, así como sus funciones y contenidos.

    Solución: *Por un lado tenemos el* main stack*, donde se almacena la pila del hilo principal, así como las interrupciones cuando son disparadas. Por otro lado están los stacks de tarea, donde cada tarea almacena su pila. Así, se puede dimensionar la pila principal para el peor caso donde se anidasen todas las interrupciones a la vez, y las pilas de los hilos hacerse mucho más pequeñas para almacenar solo el stack de funciones correspondiente, sin preocuparse de memoria extra para interrupciones.*